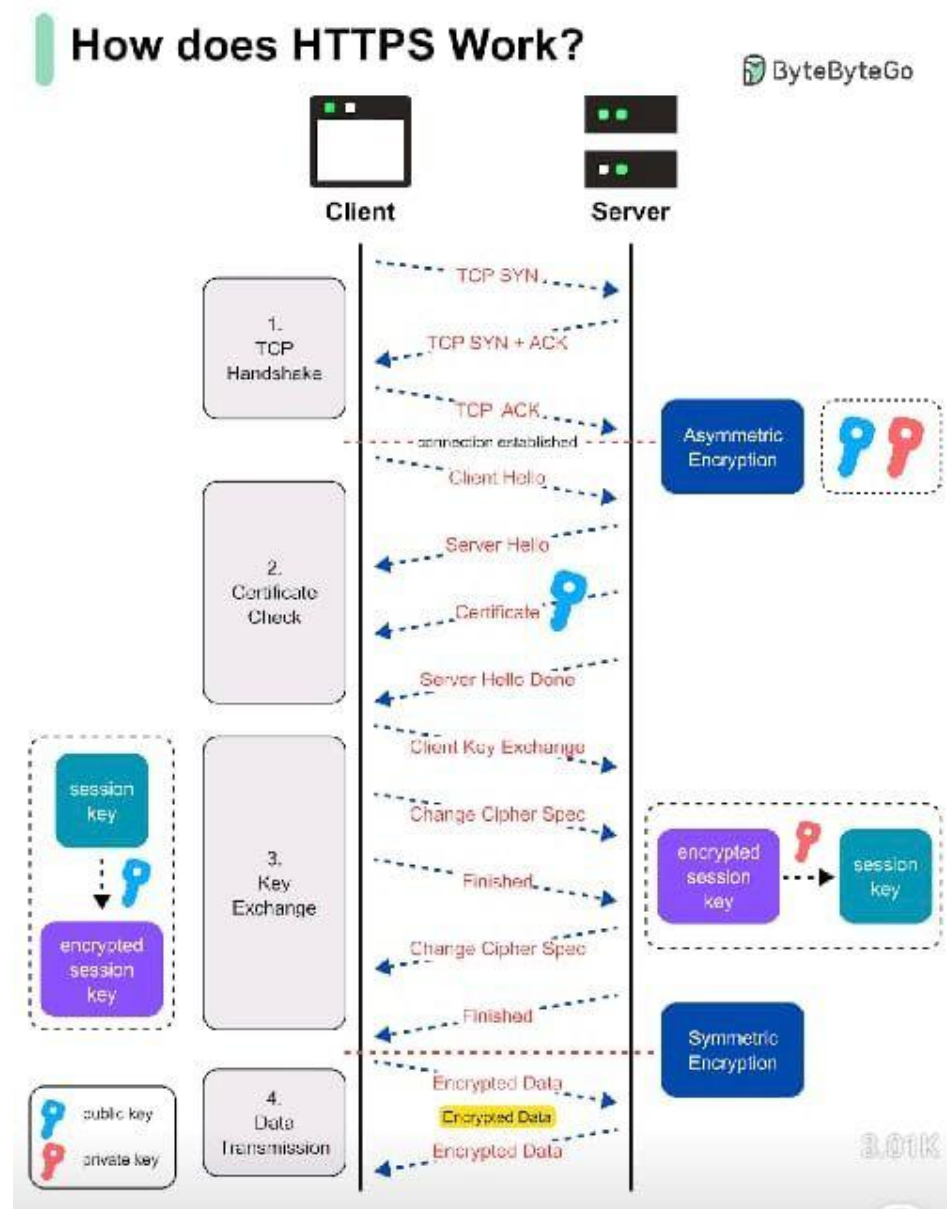
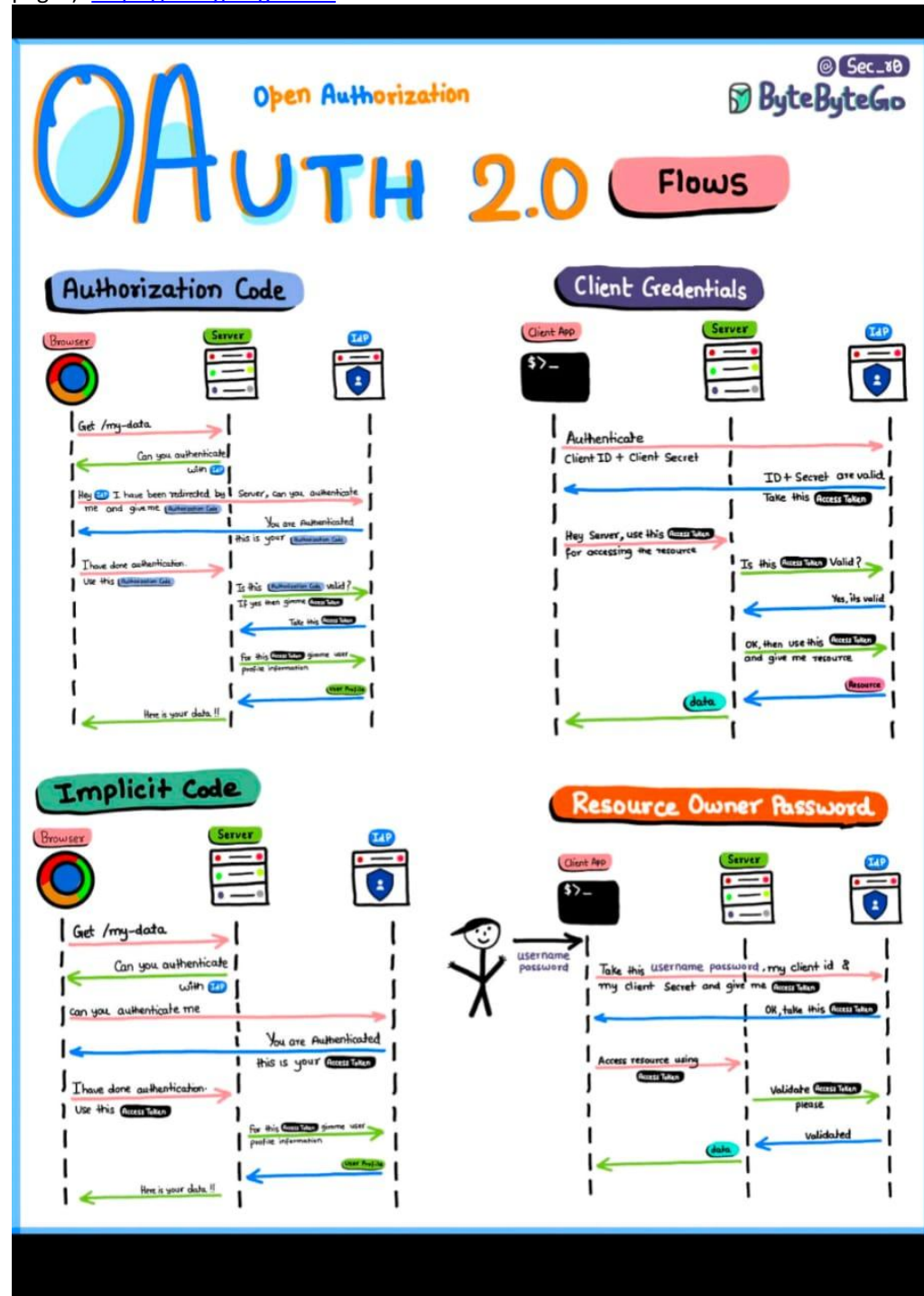


How does HTTPS work? Hypertext Transfer Protocol Secure (HTTPS) is an extension of the Hypertext Transfer Protocol (HTTP.) HTTPS transmits encrypted data using Transport Layer Security (TLS.) If the data is hijacked on-line, all the hijacker gets is binary code. How is the data encrypted and decrypted? Step 1 - The client (browser) and the server establish a TCP connection. Step 2 - The client sends a "client hello" to the server. The message contains a set of necessary encryption algorithms (cipher suites) and the latest TLS version it can support. The server responds with a "server hello" so the browser knows whether it can support the algorithms and TLS version. The server then sends the SSL certificate to the client. The certificate contains the public key, host name, expiry dates, etc. The client validates the certificate. Step 3 - After validating the SSL certificate, the client generates a session key and encrypts it using the public key. The server receives the encrypted session key and decrypts it with the private key. Step 4 - Now that both the client and the server hold the same session key (symmetric encryption), the encrypted data is transmitted in a secure bi-directional channel. Why does HTTPS switch to symmetric encryption during data transmission? There are two main reasons: 1. Security: The asymmetric encryption goes only one way. This means that if the server tries to send the encrypted data back to the client, anyone can decrypt the data using the public key. 2. Server resources: The asymmetric encryption adds quite a lot of mathematical overhead. It is not suitable for data transmissions in



OAuth 2.0 Flows Authorization Code Flow: The most common OAuth flow. After user authentication, the client receives an authorization code and exchanges it for an access token and refresh token. **Client Credentials Flow:** Designed for single-page applications. The access token is returned directly to the client without an intermediate authorization code. **Implicit Code Flow:** Designed for single-page applications. The access token is returned directly to the client without an intermediate authorization code. **Resource Owner Password Grant Flow:** Allows users to provide their user name and password directly to the client, which then exchanges them for an access token. Over to you - So which one do you think is something that you should use next in your application? Subscribe to our weekly newsletter to get a Free System Design PDF (158 pages): <https://bit.ly/bbg-social>



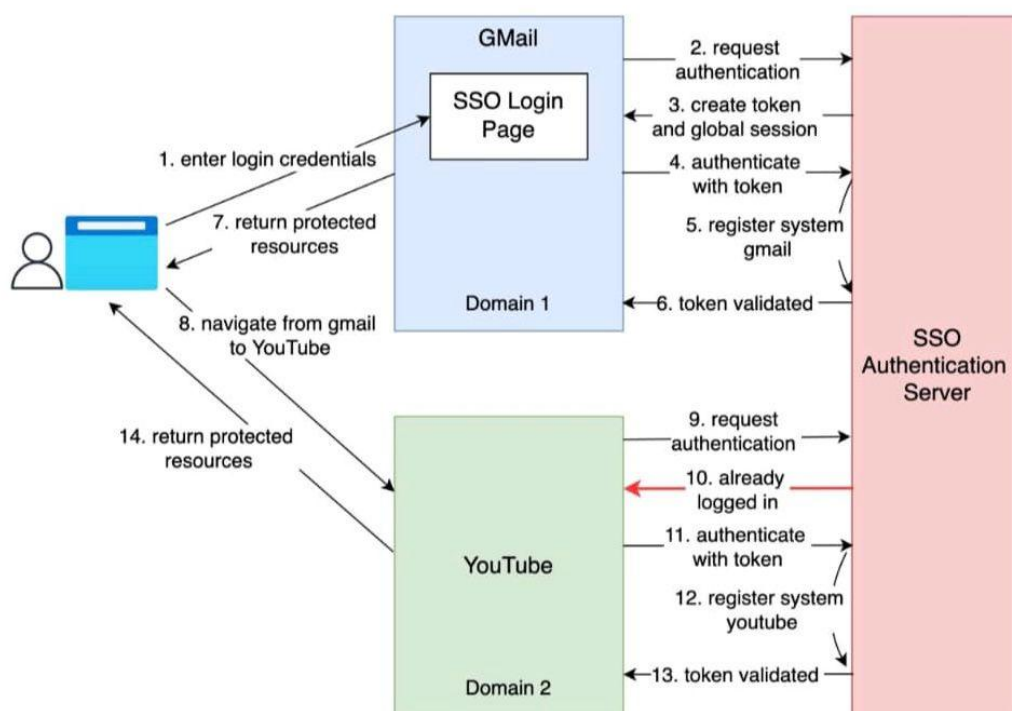
What is SSO (Single Sign-On)? Basically, Single Sign-On (SSO) is an authentication scheme. It allows a user to log in to different systems using a single ID. The diagram below illustrates how SSO works. Step 1: A user visits G mail, or any email service. Gmail finds the user is not logged in and so redirects them to the SSO authentication server, which also finds the user is not logged in. As a result, the user is redirected to the SSO login page, where they enter their login credentials. Steps 2-3: The SSO authentication server validates the credentials, creates the global session for the user, and creates a token. Steps 4-7: Gmail validates the token in the SSO authentication server. The authentication server registers the Gmail system, and returns "valid." Gmail returns the protected resource to the user. Step 8: From Gmail, the user navigates to another Google-owned website, for example, YouTube. Steps 9-10: YouTube finds the user is not logged in, and then requests authentication. The SSO authentication server finds the user is already logged in and returns the token. Step 11-14: YouTube validates the token in the SSO authentication server. The authentication server registers the YouTube system, and returns "valid." YouTube returns the protected resource to the user. The process is complete and the user gets back access to their account. Over to you

: Question 1: have you implemented SSO in your projects? What is the most difficult part?

Question 2: what's your favorite sign-in method and why

How does SSO Work?

 blog.bytebytego.com

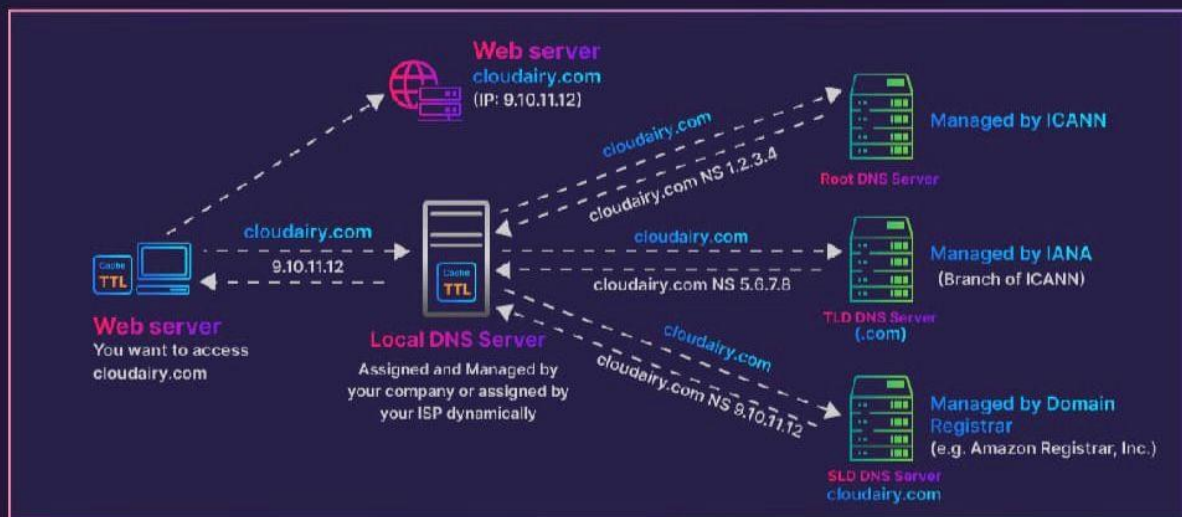


How DNS Works ! The internet is a vast and complex system, with millions of computers and devices connected to it. But how do these devices find each other? How do you know that when you type "www.cloudairy.com" into your web browser, you'll be taken to the right place? The Domain Name System (DNS) is the answer to these questions. DNS is a hierarchical naming system that translates human-readable domain names into machine-readable IP addresses. This allows us to use easy-to-remember names like "www.cloudairy.com" instead of having to remember long strings of numbers. DNS is essential for the internet to function, and it helps to make the internet more secure. But how does DNS work? Let's understand this with an example! When you type a domain name

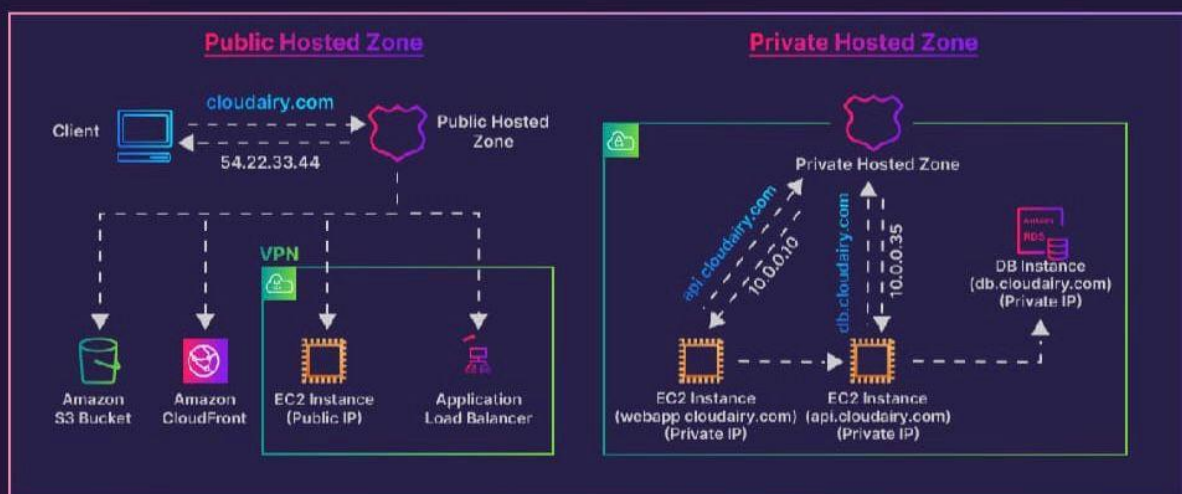
into your web browser, such as " www.cloudairy.com ", your computer sends a DNS query to a DNS resolver. The resolver then queries a series of DNS servers, starting with the root servers, to find the IP address associated with the domain name. The resolver checks its local cache to see if the IP address for the requested domain is already stored. If it finds a match, the process is complete, and the IP address is used. If the resolver doesn't have the IP address in its cache, it initiates a recursive query to find the IP address. It sends the query to a DNS root server. The root servers are the highest level in the DNS hierarchy. They contain the IP addresses of the top-level domain (TLD) servers, such as .com, .org, and .net. The TLD servers then point to the nameservers for specific domains. The nameservers store the IP addresses for all of the hosts within a domain. When the resolver receives the IP address from the nameserver, it returns it to your computer, which then connects to the host.

Resource records (RRs) are the data structures that are stored in DNS servers. There are many different types of RRs, but some of the most common ones are A records (which store IPv4 addresses), AAAA records (which store IPv6 addresses), MX records (which store mail servers), and CNAME records (which store aliases).

How DNS works??



Route 53 - Public vs. Private Hosted Zones



Types of Malware Viruses:

Malicious code attached to files that spread through sharing; can be harmless or destructive.

- Worms: Self-replicating malware that spreads across networks without needing a host program.

- Trojan Horses: Malware disguised as legitimate software, attached to non-executable files.

Ransomware: Encrypts data and demands ransom for decryption; access is lost without the key.

- Adware: Displays unwanted ads and pop-ups, generating revenue for its distributor.

Spyware: Steals personal data and transmits it to hackers.

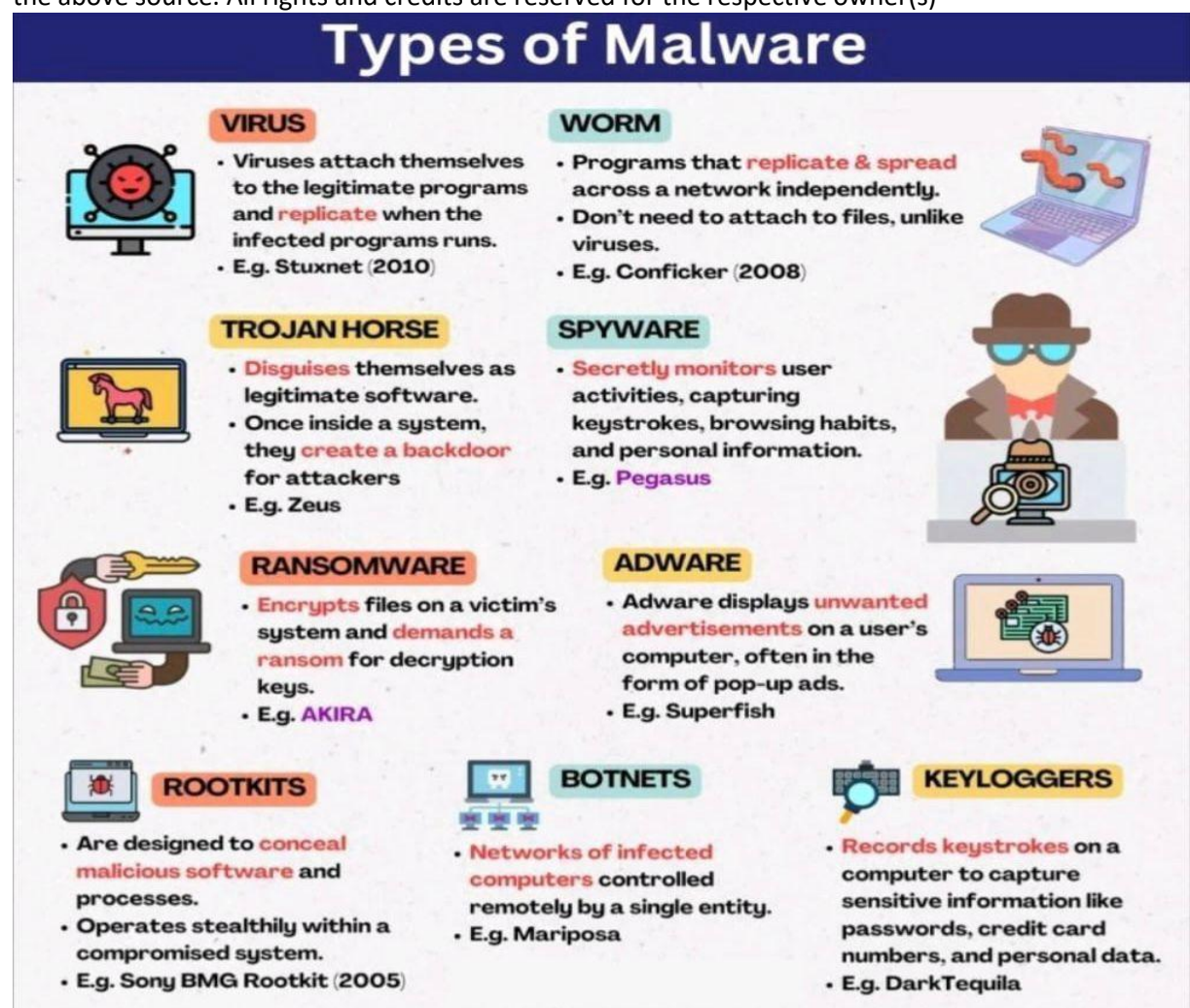
- Logic Bombs: Dormant until triggered, executing malicious actions, sometimes destroying hardware.

- Rootkits: Modify operating systems to create backdoors for remote access to attackers.

- Backdoors: Bypass authentication, allowing ongoing access to cybercriminals.

- Keyloggers: Record keystrokes to capture sensitive information, including passwords.

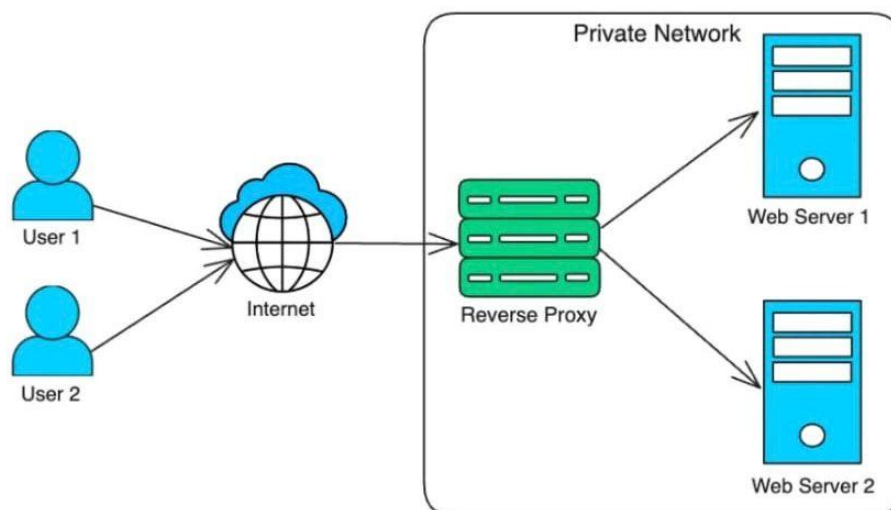
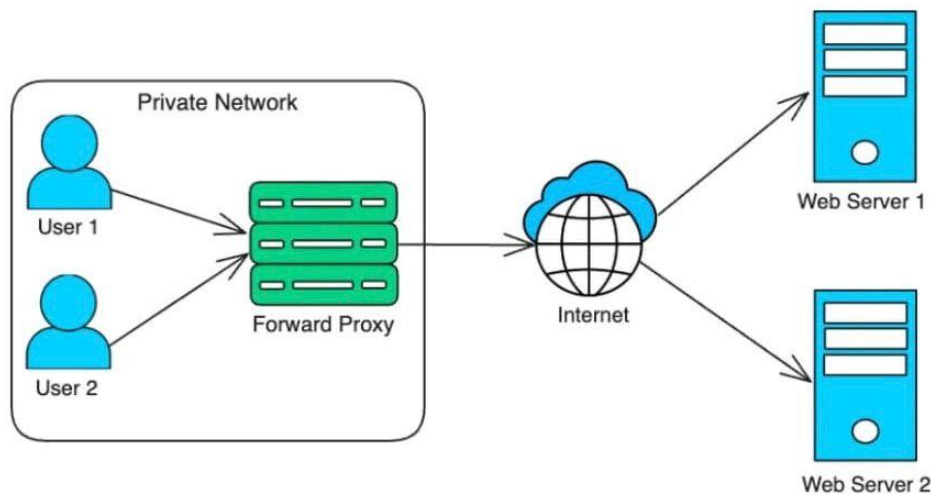
Image Source: Unknown Disclaimer - This post has only been shared for an educational and knowledge-sharing purpose related to technologies. Image Information was obtained from the above source. All rights and credits are reserved for the respective owner(s)



Proxy vs Reverse Proxy - What's the difference? • Proxy A proxy server (sometimes called a Forward proxy) acts as an intermediary between a client and the internet. It forwards client requests to the destination server and then returns the server's response to the client. Proxies are client-focused, acting on behalf of the user. • Use cases: - Anonymity: Hides client's IP address - Access Control: Restricts website access - Content Filtering: Blocks malicious sites - Caching: Improves performance • Reverse Proxy A reverse proxy sits in front of web servers and forwards client requests to appropriate backend servers. It hides the server's identity from the client and can distribute the load across multiple servers. Reverse proxies are server-focused, acting on behalf of the backend servers. Use cases: - Load Balancing: Distributes traffic across servers SSL Termination: Handles HTTPS encryption/decryption - Caching: Reduces load on backend servers - Security: Hides server infrastructure details

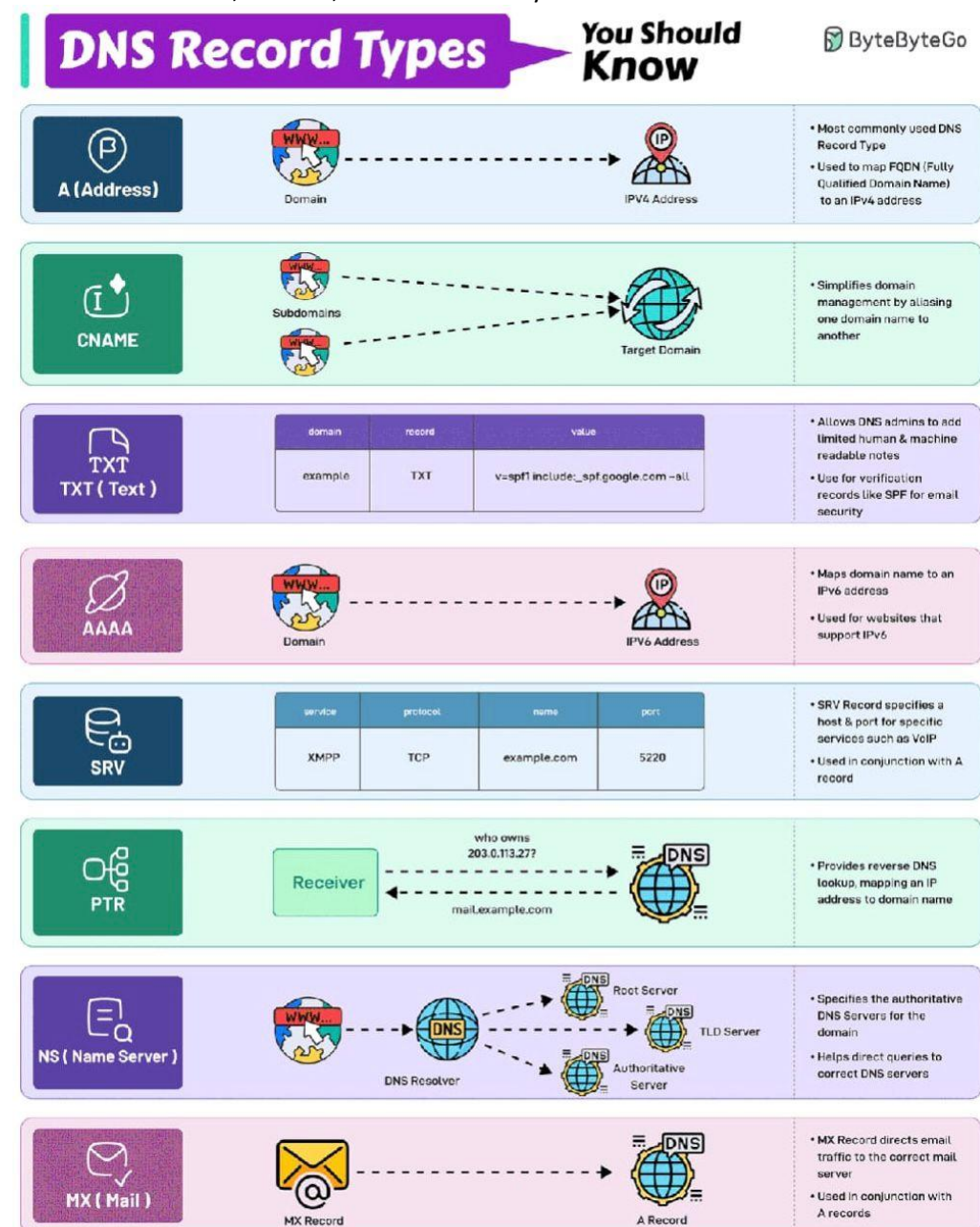
Proxy vs Reverse Proxy

blog.algomaster.io



DNS Record Types You Should Know! ... Here are the 8 most commonly used DNS Record Types.

- 1 - A (Address) Record Maps a domain name to an IPv4 address. It is one of the most essential records for translating human-readable domain names into IP addresses.
- 2 - CNAME (Canonical Name) Record Used to alias one domain name to another. Often used for subdomains, pointing them to the main domain while keeping the actual domain name hidden.
- 3 - AAAA Record Similar to an A record but maps a domain name to an IPv6 address.
- 4 - PTR Record Provides reverse DNS lookup, mapping an IP address back to a domain name. It is commonly used in verifying the authenticity of a server.
- 5 - MX Record Directs email traffic to the correct mail server.
- 6 - NS (Name Server) Record Specifies the authoritative DNS servers for the domain. These records help direct queries to the correct DNS servers for further lookups.
- 7 - SRV (Service) Record SRV record specifies a host and port for specific services such as VoIP. They are used in conjunction with A records.
- 8 - TXT (Text) Record Allows the administrator to add human-readable text to the DNS records. It is used to include verification records, like SPF, for email security.



How Single Sign on(SSO)/SAML Works ! AssumeRole: Easily switch roles within your AWS account or between accounts for straightforward permission management. AssumeRoleWithSAML: Use SAML (a secure standard) to get credentials for users, making sure only the right people have access. AssumeRoleWithWebIdentity: For users signing in via Amazon Cognito, Google, or Facebook, AWS suggests using Cognito for a hassle-free login. GetSession Token: Boost security by adding Multi-Factor Authentication (MFA) for users or the main AWS account holder. GetFederation Token: Great for apps acting on behalf of users, this gives temporary access to those who need it, keeping things flexible yet secure. Identity Federation in AWS: This lets you extend AWS access to users outside of AWS, without having to individually manage each user. It's great for organizations that already have their own identity systems (like Active Directory) or for apps that need to use AWS resources. Key features include: SAML 2.0 Integration: Works smoothly with well-known identity providers, granting temporary access to AWS services. Custom Identity Broker: A solution for when your identity provider isn't SAML-compatible, managing authentication and AWS access.

